Sensor and Systems
Simple sensors

Jan Bendtsen, Torben Knudsen

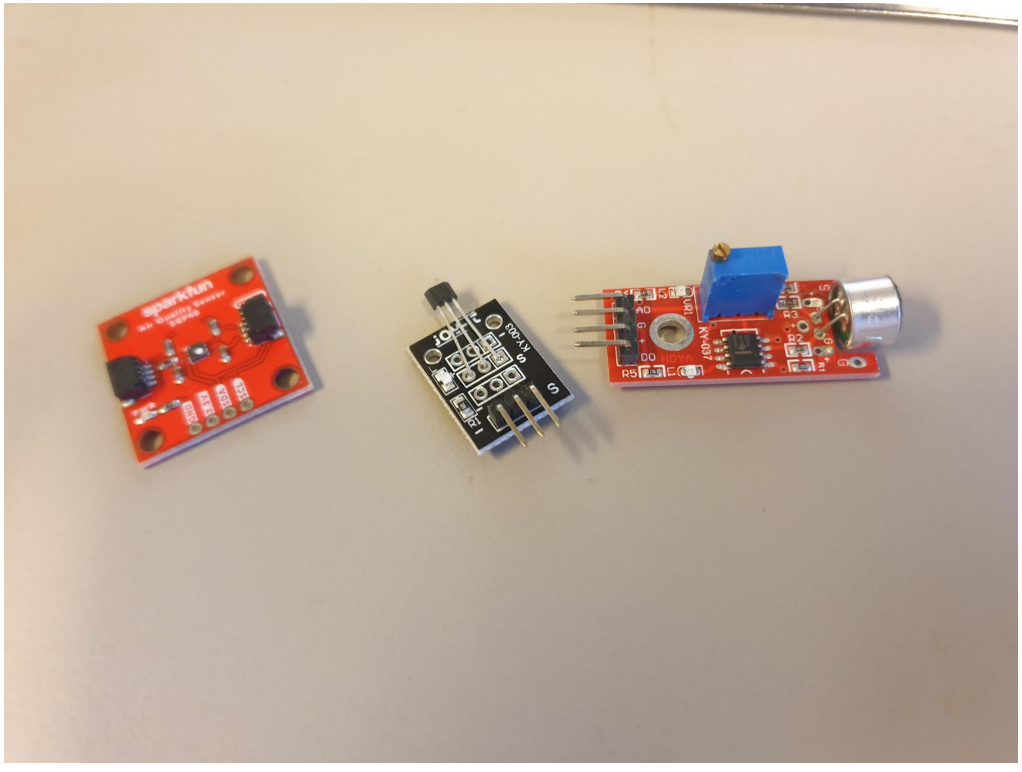
Electronic Systems

8th semester

Overview

- Simple sensors
  - Hall sensor
  - Temperature sensor
  - Humidity sensor
- Measurement noise
  - Gaussian noise
  - Thermal, flicker and shot noise
- Simple filtering

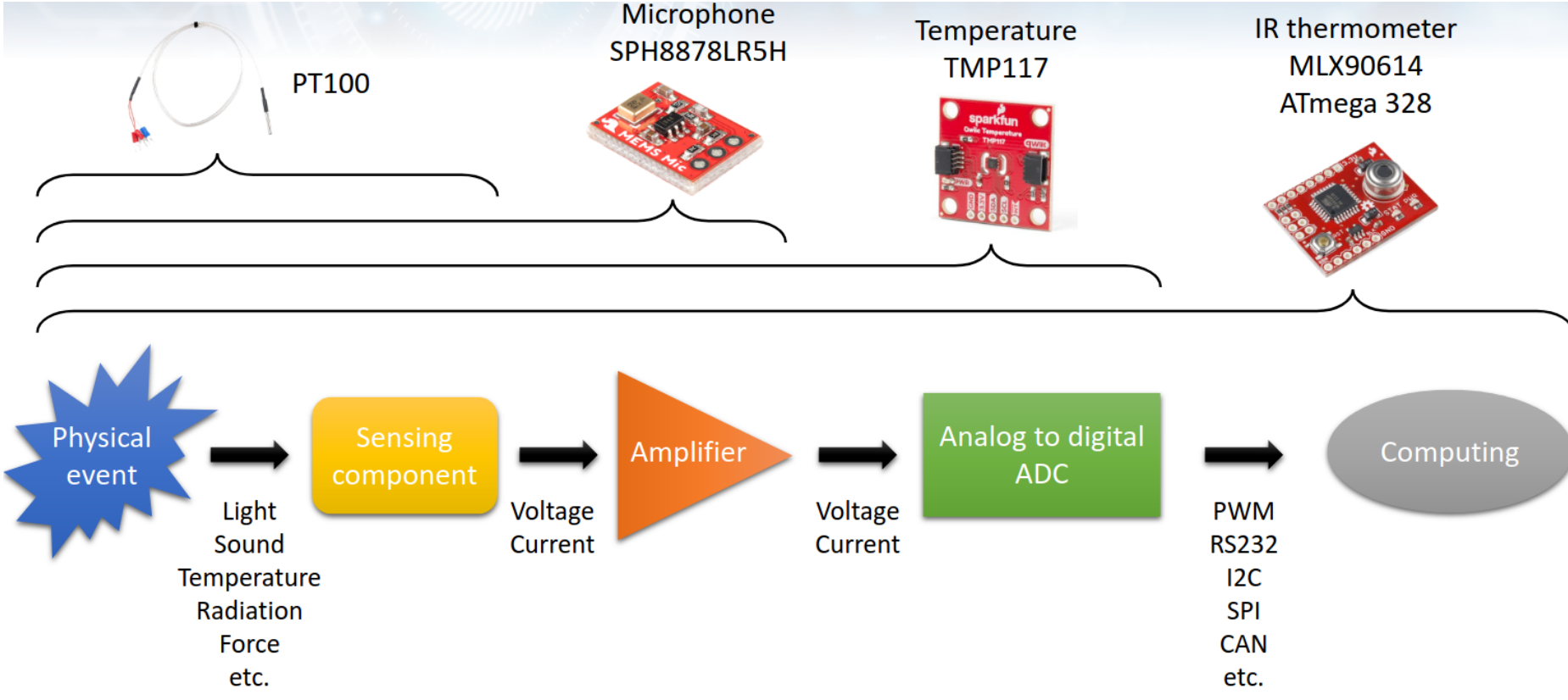
Simple sensors



"Simple" sensors (From left: air quality, Hall effect, microphone)

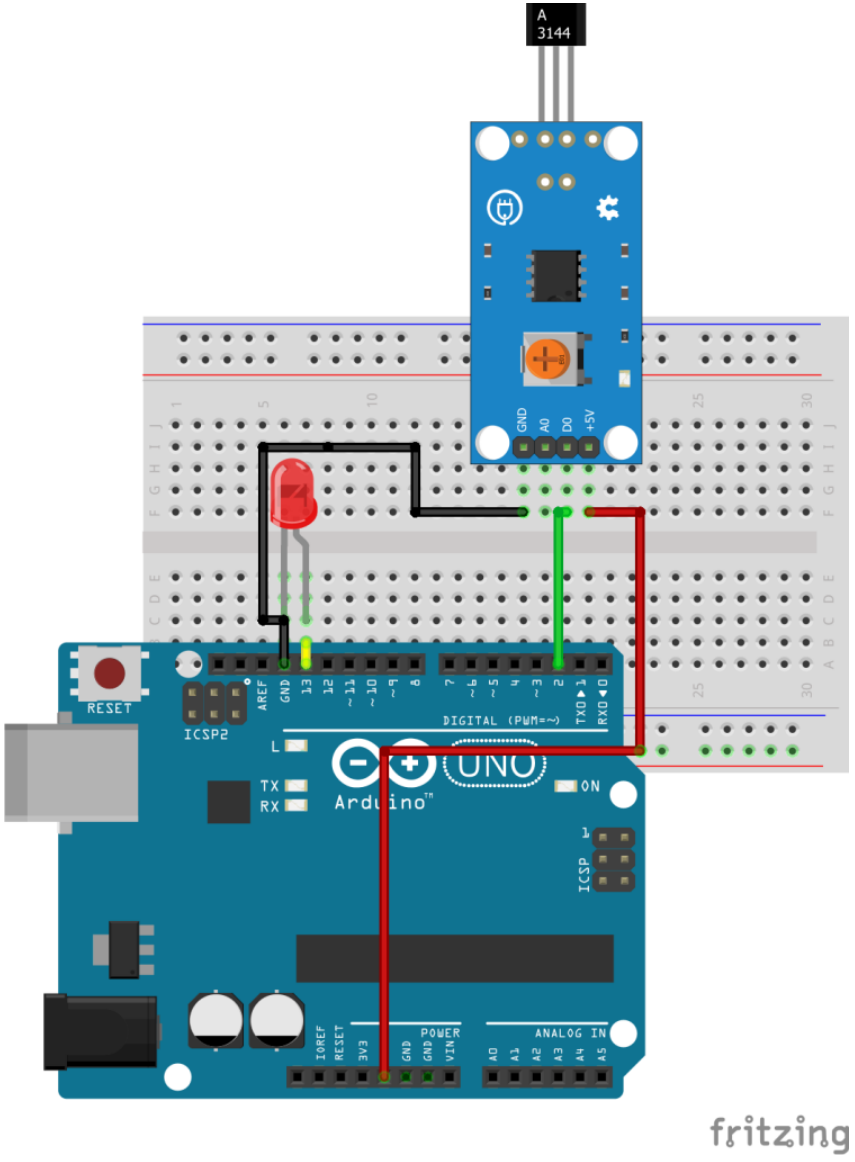
- Measure one continuous physical quantity;
- Provide a simple analog or digital interface;
- Do not require extensive processing to obtain the desired information.

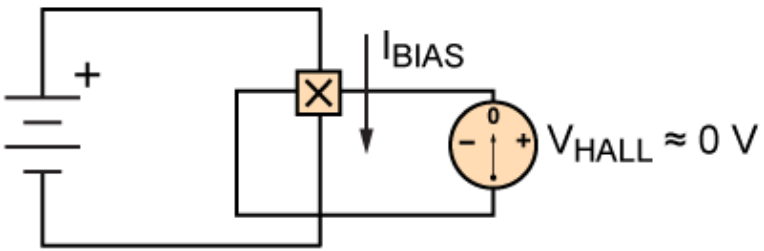
Simple sensor interfaces

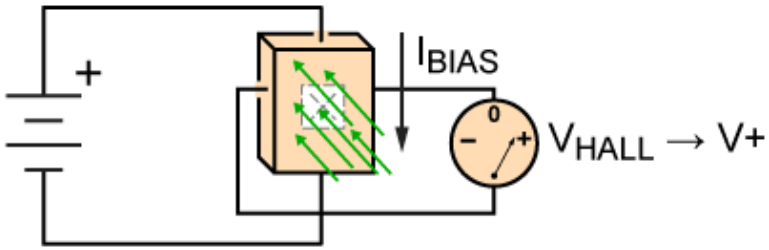


Hall sensor

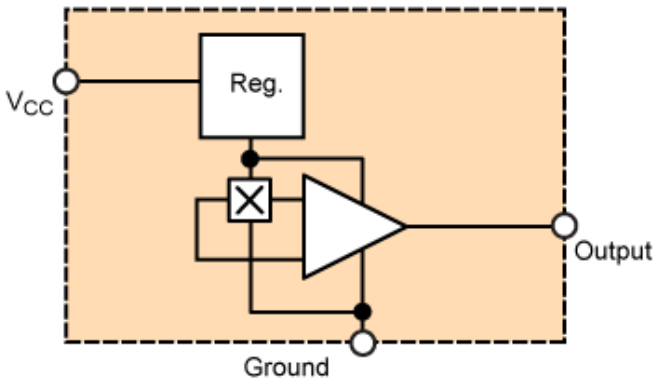

49E
11082N

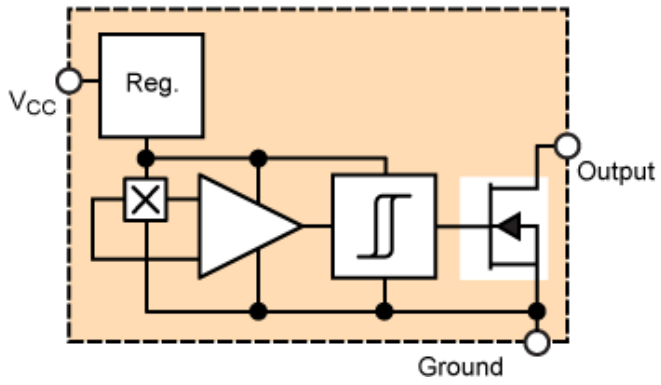

A
3144
GND
A0
DO
+5V
RESET
ICSP2
DIGITAL (PWM=~)
TX
RX
ON
ICSP
POWER
ANALOG IN
fritzing





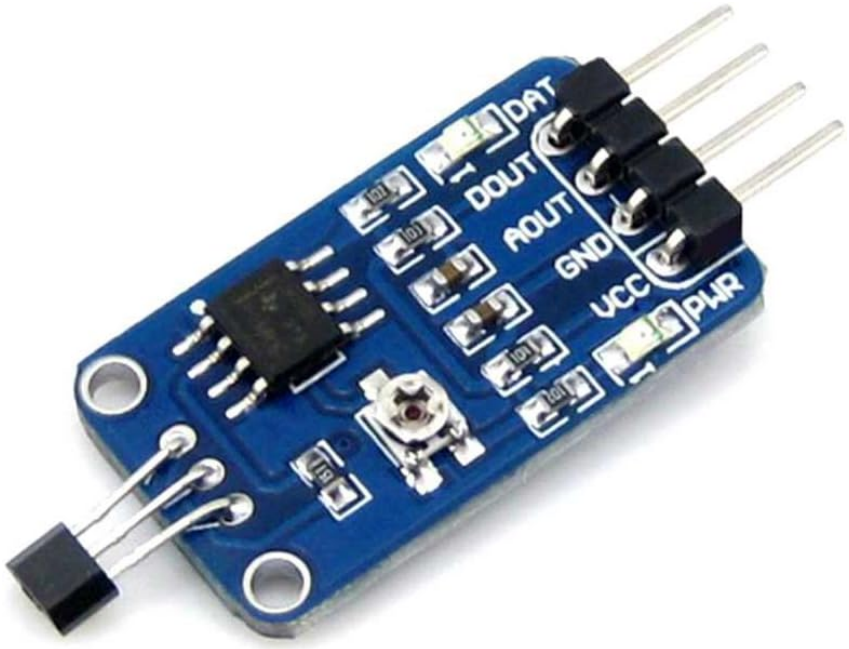
- A constant voltage source forces a constant bias current, I_{BIAS}, to flow in the semiconductor sheet (marked with $\times$).
- The output takes the form of a voltage, V_{HALL}, measured across the width of the sheet.
- If the biased Hall element is placed in a magnetic field with flux lines at right angles to the bias current, the voltage output changes in direct proportion to the strength of the magnetic field.





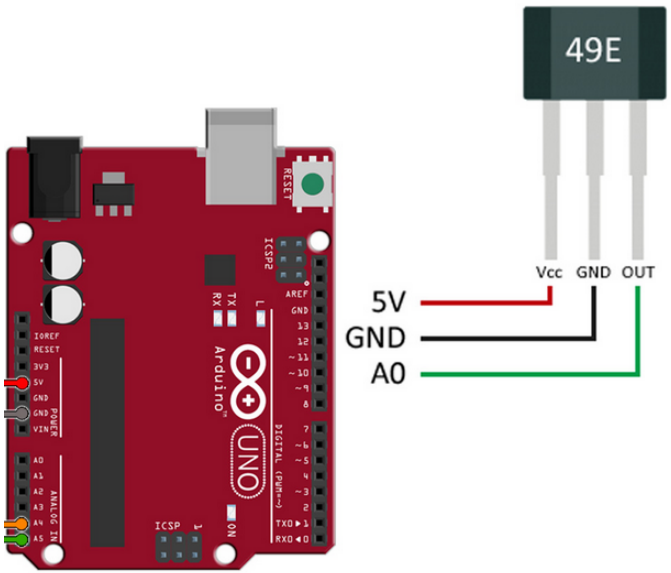
- V_{HALL} is typically very small, so an OpAmp is connected to provide amplification. This configuration allows measuring *field strength*, but requires a high-quality OpAmp.
- If one is only interested in *detecting* the presence of a magnetic field, a detector circuit is connected after the amplifier.

Hall sensor
SS49E



- Sensitivity: 2.5±0.5mV/G
- Power consumption: 4 mA at 5 V DC
- Can operate down to 2.3 V

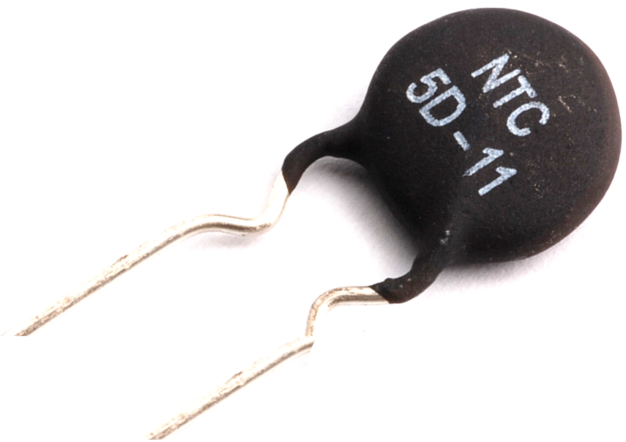
Hall sensor
Example of usage

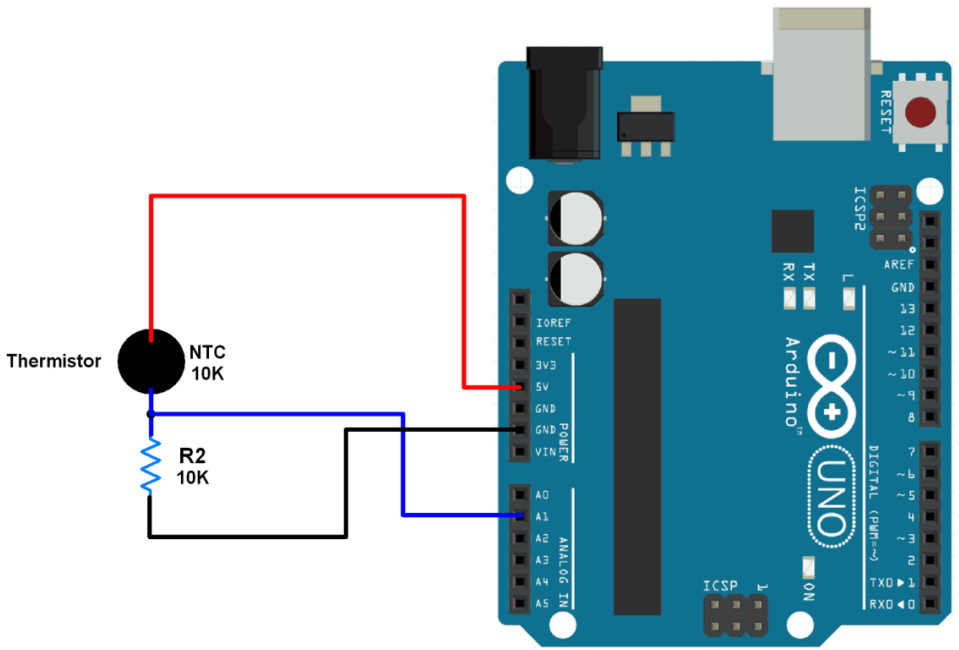



```
const int pinHall = A0;
void setup() {
  pinMode(pinHall, INPUT);
  Serial.begin(9600);
}
void loop() {

  //we measure 10 times adn make the mean
  long measure = 0;
  for(int i = 0; i < 10; i++){
      int value =
      measure += analogRead(pinHall);
  }
  measure /= 10;
  //voltage in mV
  float outputV = measure * 5000.0 / 1023;
  Serial.print("Output Voltaje = ");
  Serial.print(outputV);
  Serial.print(" mV   ");

  //flux density
  float magneticFlux =  outputV * 53.33 - 133.3;
  Serial.print("Magnetic Flux Density = ");
  Serial.print(magneticFlux);
  Serial.print(" mT");
  delay(2000);
}
```

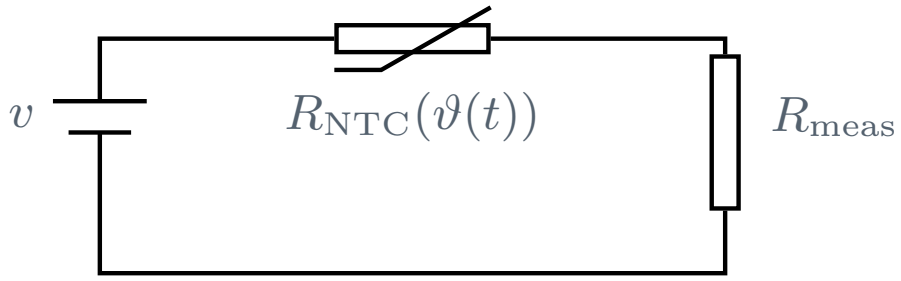
Temperature sensor

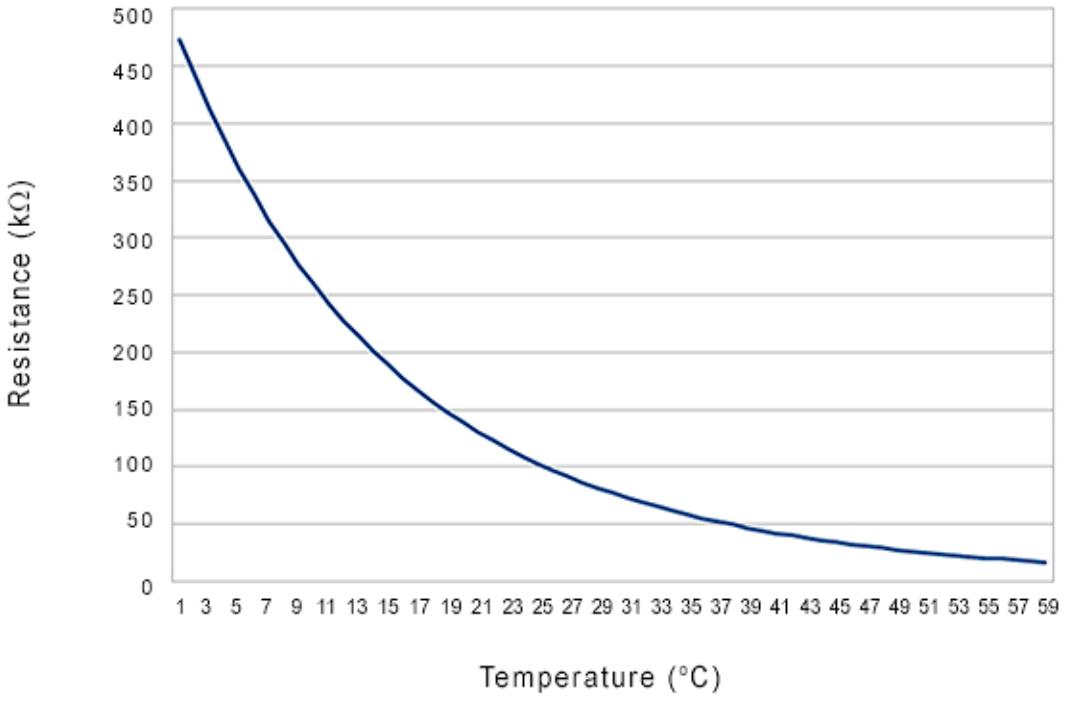
Negative Temperature Coefficient (NTC) thermistor





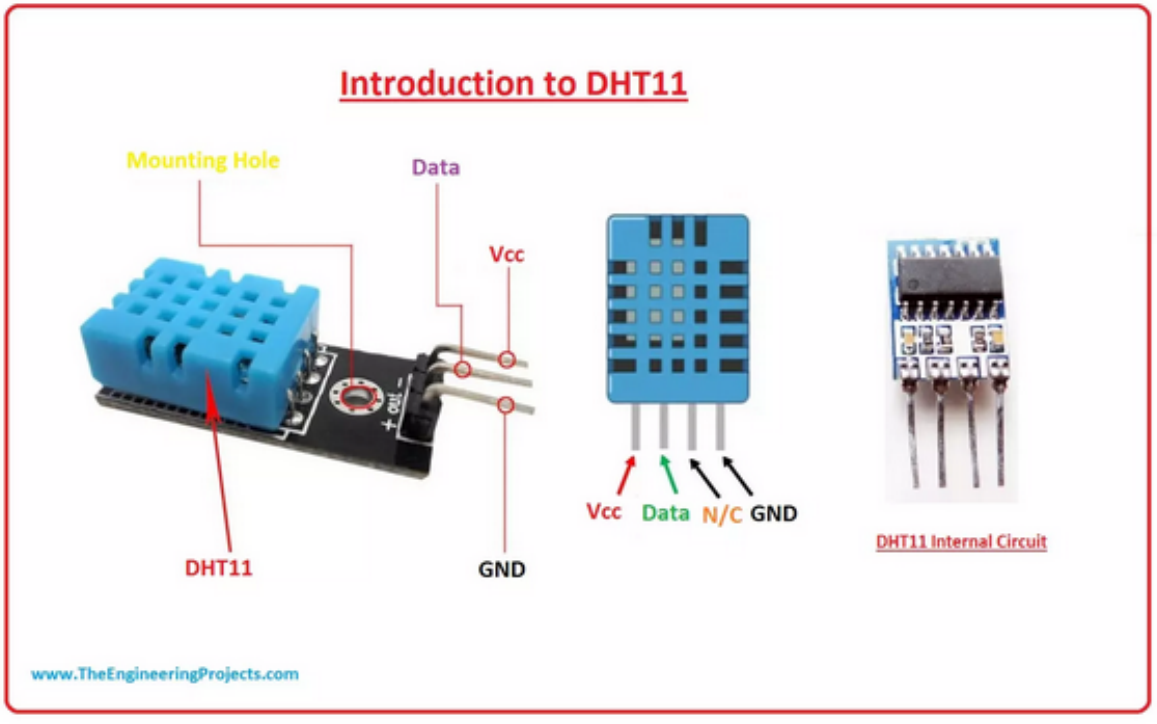
Temperature-dependent resistance



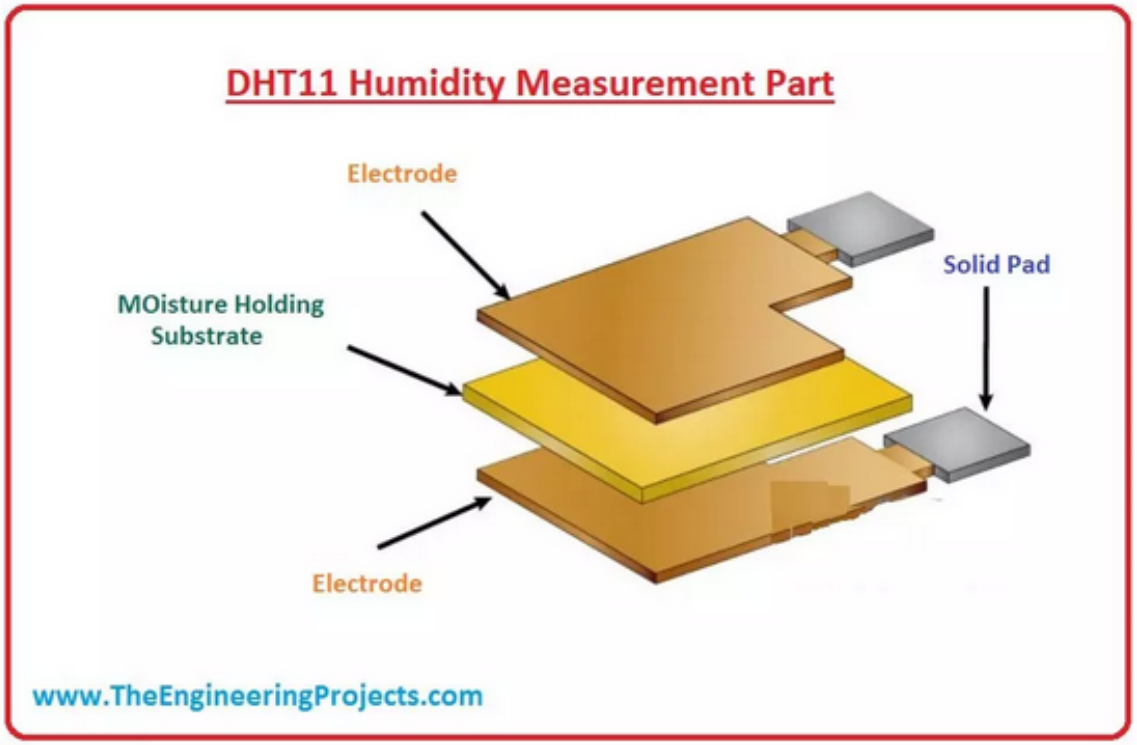


Kilde: `https://www.teamwavelength.com/info/thermistors.php`

Combined temperature and humidity sensor
DHT11

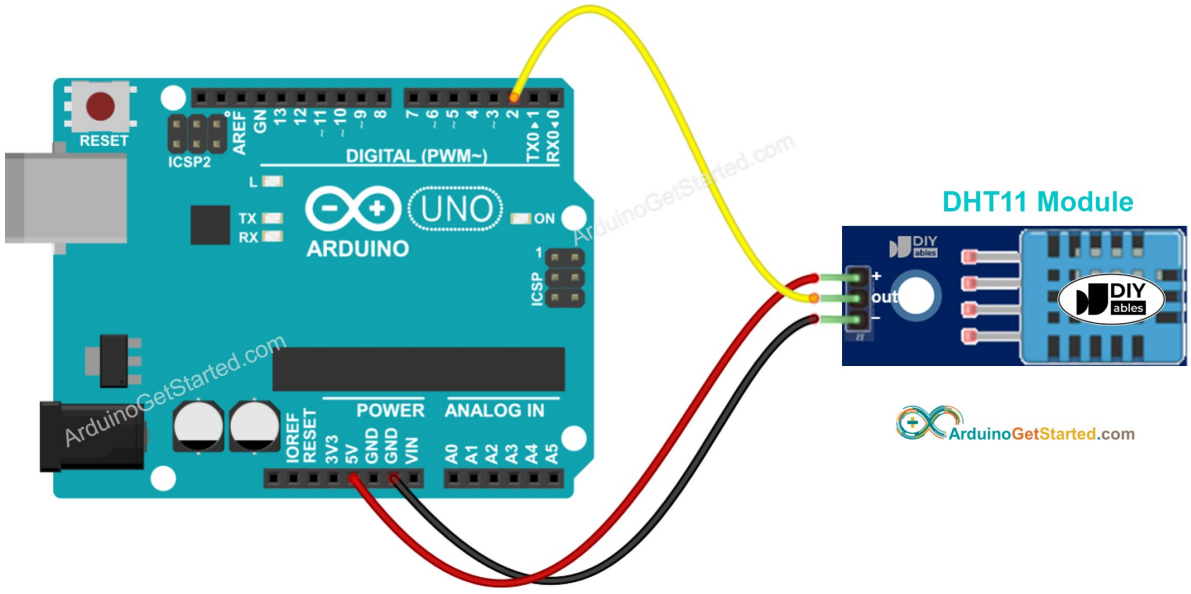


- Humidity range: 20% to 80% with ±5% accuracy
- Temperature range: 0°C to 50°C with ±2°C accuracy
- Operating voltage/currency:3.0V to 5.5V / 0.3mA (measuring), 60μA (idle)
- Sampling frequency: 1Hz
- Response time: 6 to 15s



- DHT11 uses a capacitor-like humidity sensor with two electrodes and a substrate material in between.
- As the environment moisture content changes, the moisture content in the substrate material follows suit, which in turn changes the capacitance between the electrodes.
- This change is then measured, similarly to the NTC.

Communicating with DHT11



The DHT11 sensor is activated by setting the data pin LOW for at least 18μs.
It then communicates using 5-byte serial data packets:

1. Humidity in % : 8-bit integer
2. Humidity, decimal part : 8 zero-bits
3. Temperature in °C : 8-bit integer
4. Temperature, decimal part : 8 zero-bits
5. 8-bit checksum

Sensor dynamics

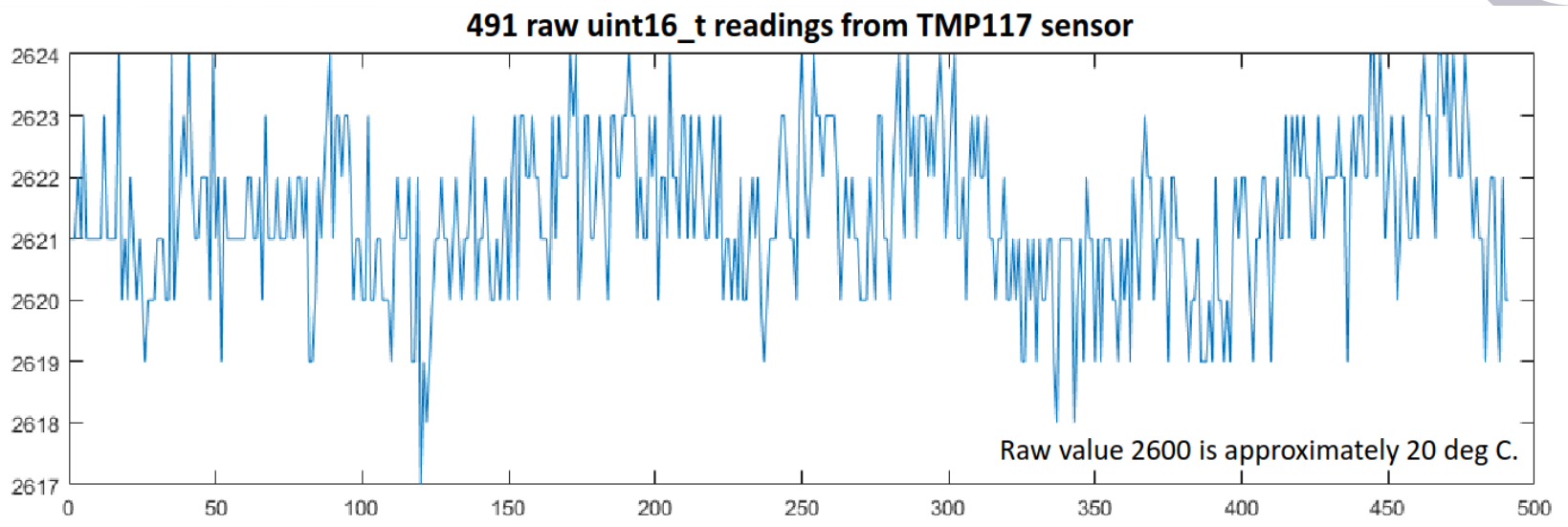
- Newton's law of cooling: *"The rate of heat loss of a body is directly proportional to the difference in the temperatures between the body and its surroundings provided the temperature difference is small and the nature of radiating surface remains the same."*
- I.e.,

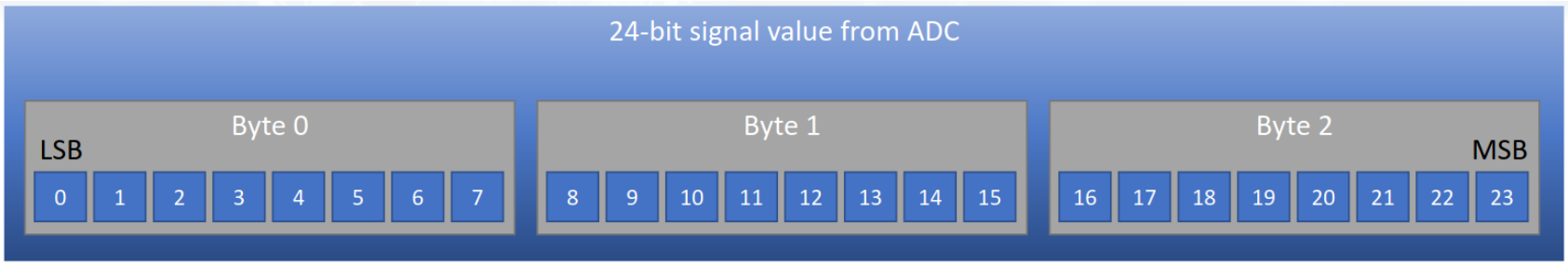
$$\frac{d\vartheta(t)}{dt} = \frac{UA}{C_p}(\vartheta_a(t) - \vartheta(t))$$

  where ϑ_a is the ambient temperature, UA is the product of surface area and heat transfer coefficient, and C_p is the sensor's heat capacity.
- 1st order model:

$$\dot{x}(t) = -\frac{UA}{C_p}x(t) + \frac{UA}{C_p}u(t), \qquad x(0) = \vartheta(0)$$
$$y(t) = \beta x(t)$$

Raw data



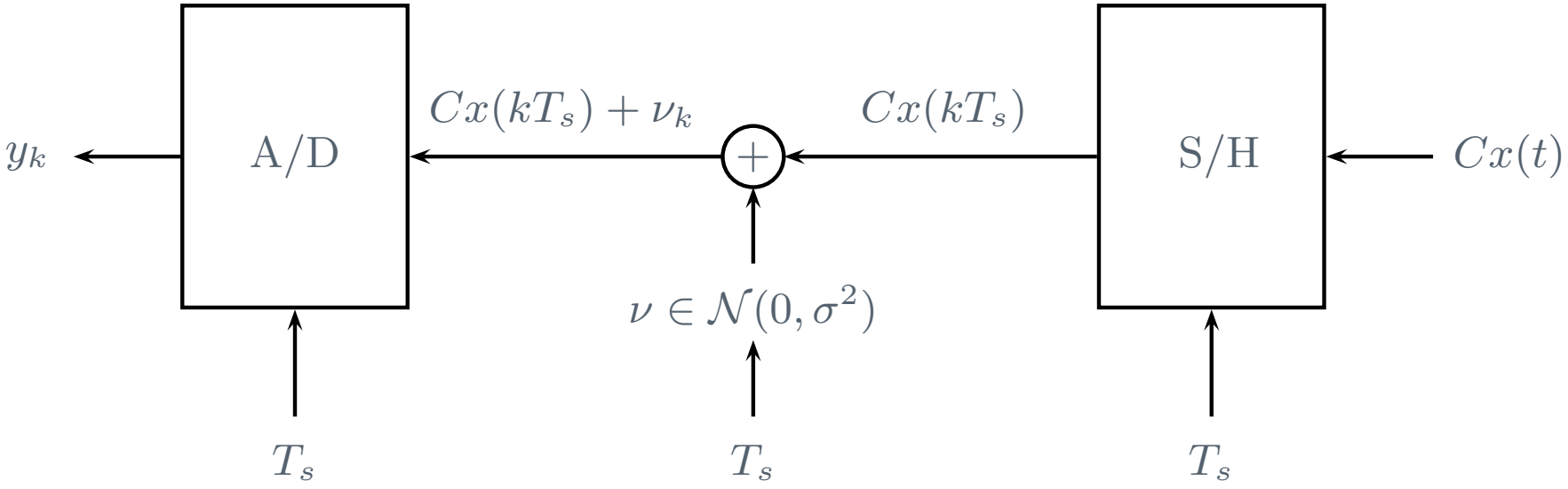


Noise floor
Defined as bits that change when the measured value is constant. Resulting from a sum of all noise.

Signal
Defined as bits that change when the measured value change.

Surplus
Defined as bits that are always 0.

Measurement noise as part of sampling

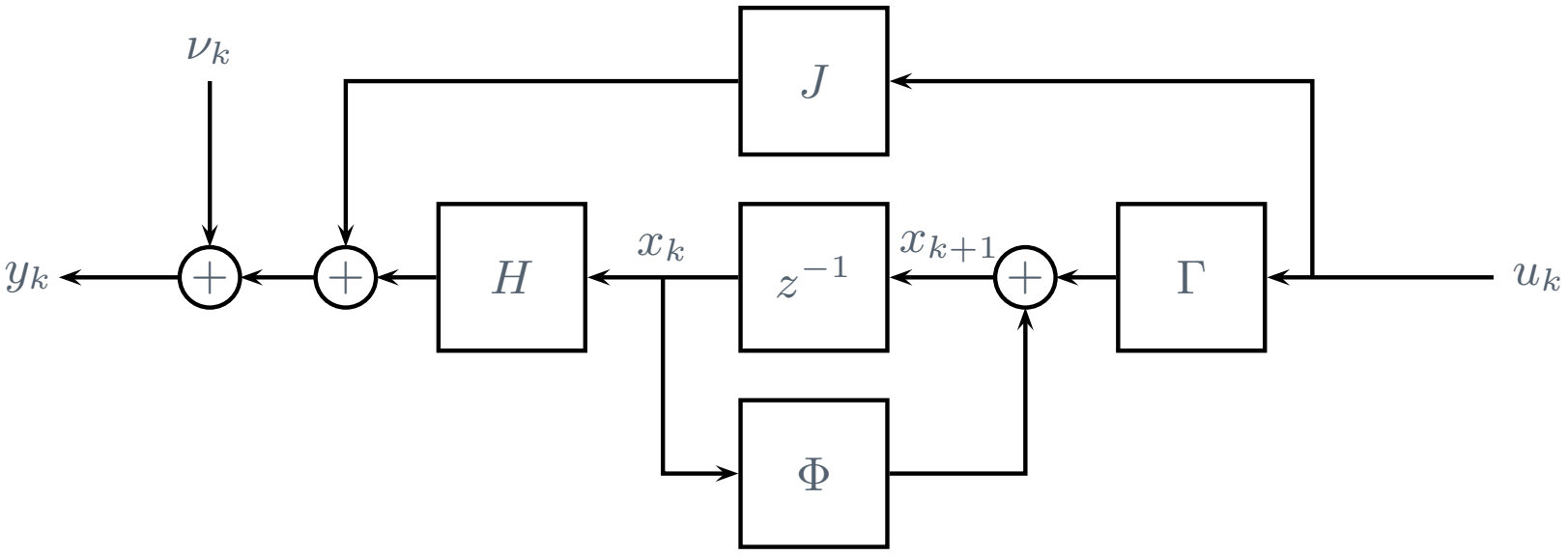


- From a modeling perspective, it is most convenient to consider measurement noise as part of the sampling process itself.
- Each 'noise sample' them only requires a countable number of random variable picks
- Continuous-time and discrete-time model of sampled signal:

$$y(t) = Cx(t) + ?$$

$$y_k = Hx_k + Ju_k + \nu_k$$

Measurement noise affecting discrete-time system



- Discrete-time model with measurement noise:

$$x_{k+1} = \Phi x_k + \Gamma u_k$$
$$y_k = H x_k + J u_k + \nu_k$$

Basic random variable statistics

- Probability distribution:

$$\Pr(a \leq X \leq b) = \int_a^b f_X(x)dx.$$

- If F_X is the *cumulative distribution* of the random variable X:

$$F_X(x) = \int_{-\infty}^{x} f_X(u)\,du,$$

  then f_X is the *probability density function* (PDF) of X

$$f_X(x) = \frac{d}{dx}F_X(x).$$

- Expectation:

$$\mathrm{E}(X) = \mu_X = \int_{-\infty}^{\infty} x f_X(x)dx$$

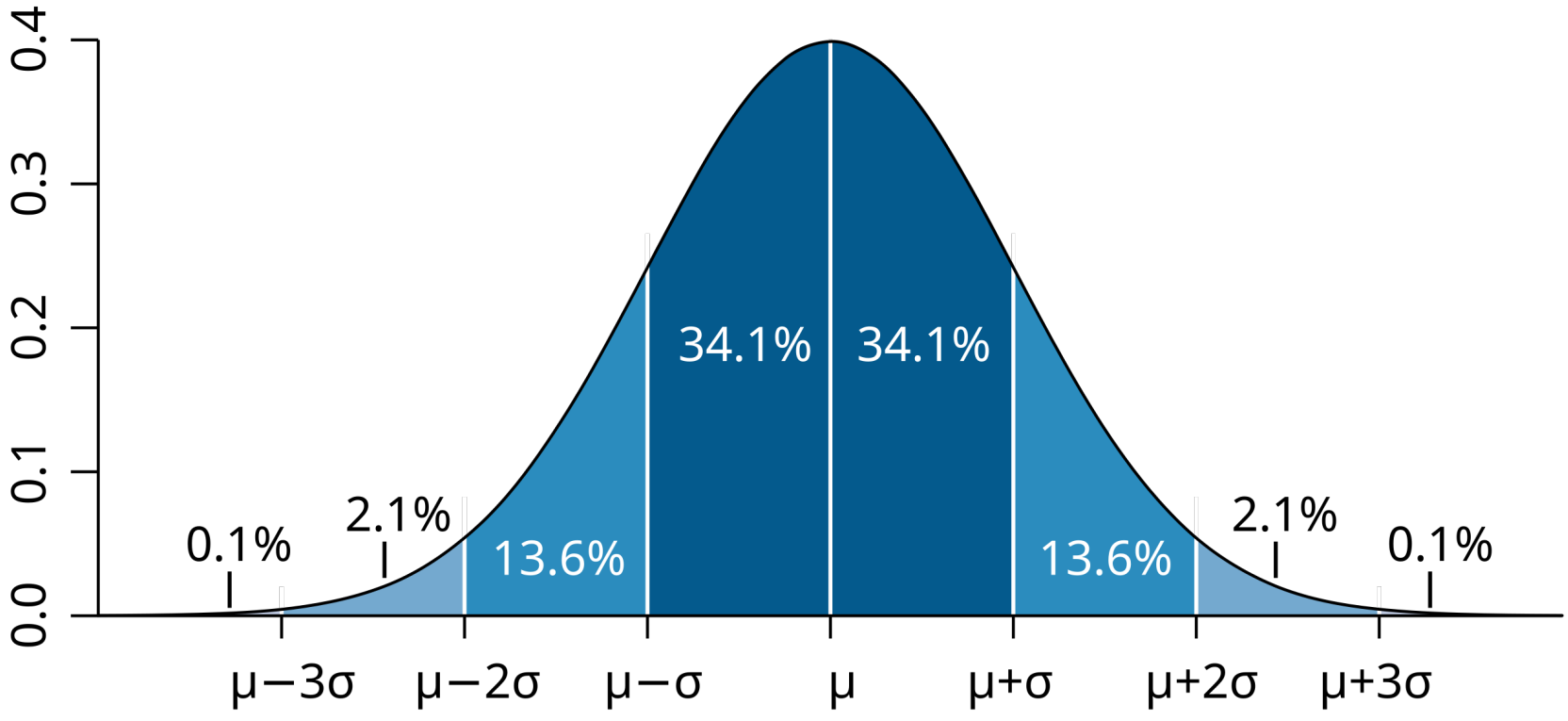
- Variance:

$$\sigma_X^2 = \mathrm{Var}(X) = \mathrm{E}\left((X - \mu_X)^2\right) = \mathrm{E}(X^2) - \mathrm{E}(X)^2.$$

- Spread:

$$\sigma_X = \sqrt{\mathrm{Var}(X)}.$$

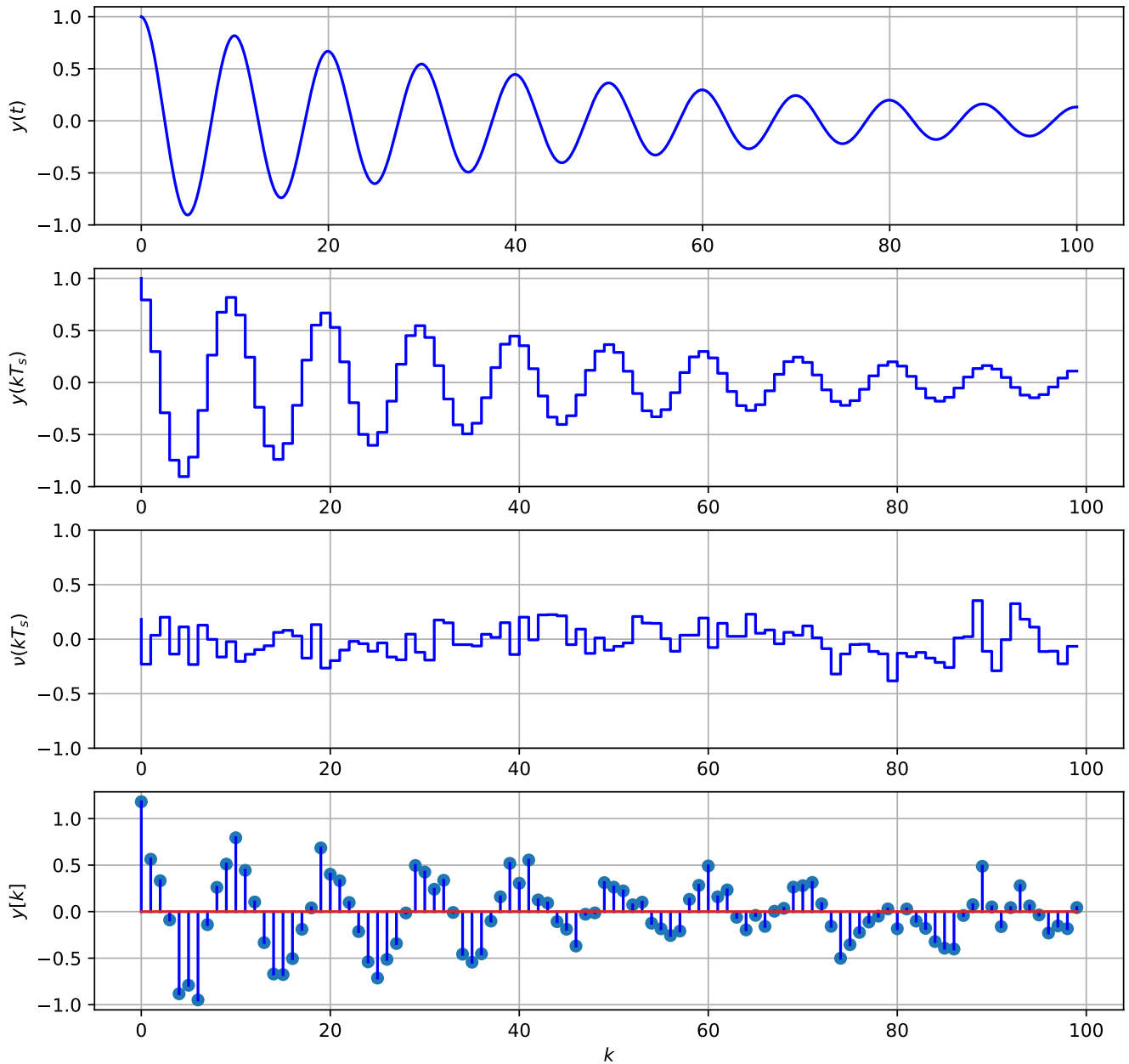
Normal distribution
Probability density function



$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Additive noise

'Nominal' signal, sample/hold, Gaussian noise, sampled noisy signal



- The *cross correlation* between two signals $y_1(t)$ and $y_2(t)$ is given as the *convolution* of y_1 with y_2:

$$R_{y_1 y_2}(\tau) = \int_{-\infty}^{\infty} y_1^*(\tau) y_2(t+\tau) d\tau = \int_{-\infty}^{\infty} y_1^*(t-\tau) y_2(\tau) d\tau$$

and in discrete time:

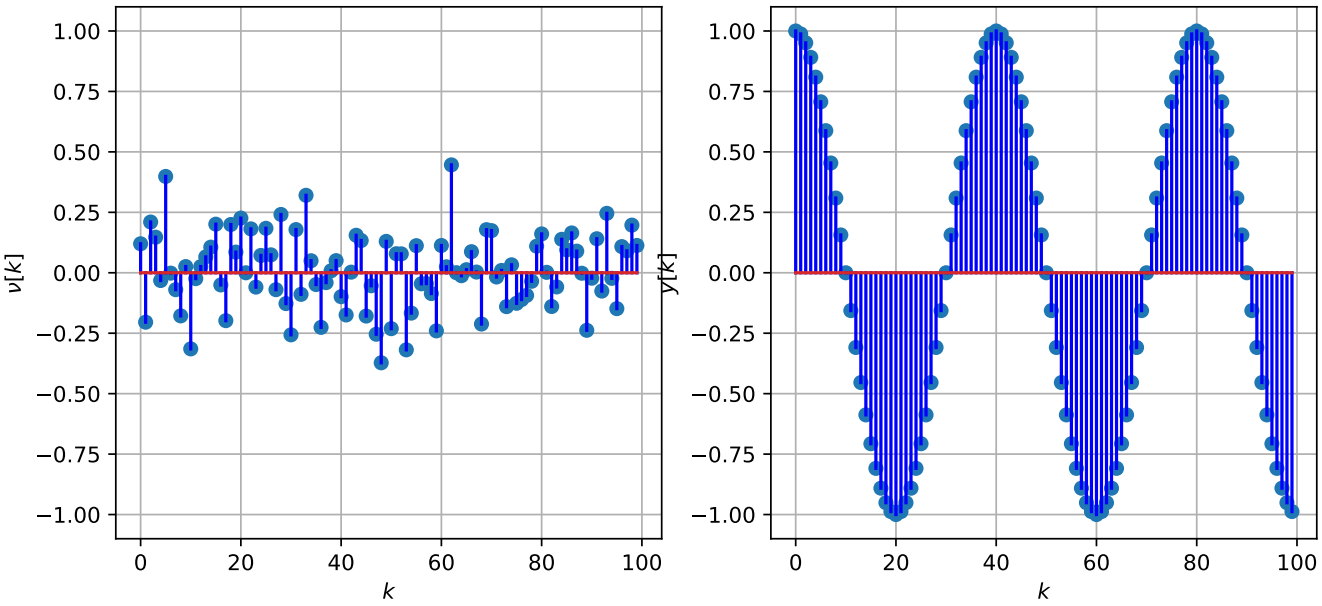
$$R_{y_1 y_2}(\kappa) = \sum_{k=-\infty}^{\infty} y_{1,k}^* y_{2,k+\kappa} = \sum_{k=-\infty}^{\infty} y_{1,k-\kappa}^* y_{2,\kappa}$$

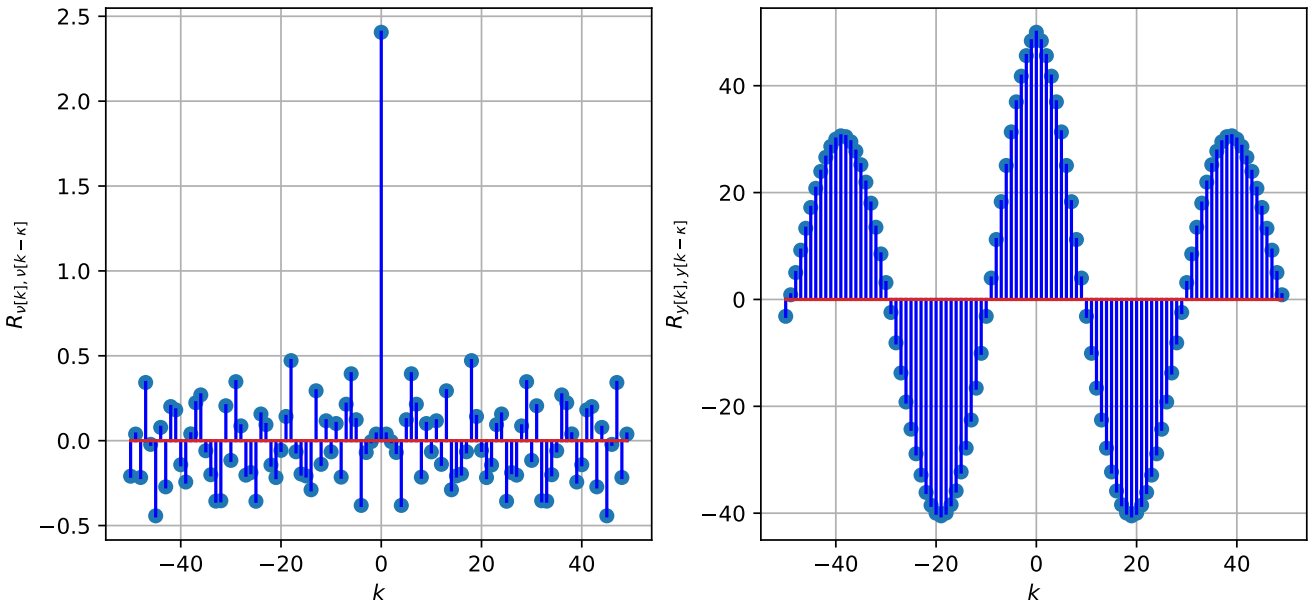
- The *autocorrelation* of a signal is the cross correlation of the signal with itself:

$$R_y(\tau) = \int_{-\infty}^{\infty} y^*(\tau) y(t+\tau) d\tau = \int_{-\infty}^{\infty} y^*(t-\tau) y(\tau) d\tau$$

and in discrete time:

$$R_y(\kappa) = \sum_{k=-\infty}^{\infty} y_k^* y_{k+\kappa} = \sum_{k=-\infty}^{\infty} y_{k-\kappa}^* y_\kappa$$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
v[k]
0
20
40
60
80
100
k
y[k]


2.5
2.0
1.5
1.0
0.5
0.0
−0.5
$R_{v[k],v[k-\kappa]}$
−40
−20
0
20
40
k
$R_{y[k],y[k-\kappa]}$

- Assuming the total energy of a signal $y(t)$ is finite, Parseval's theorem states that

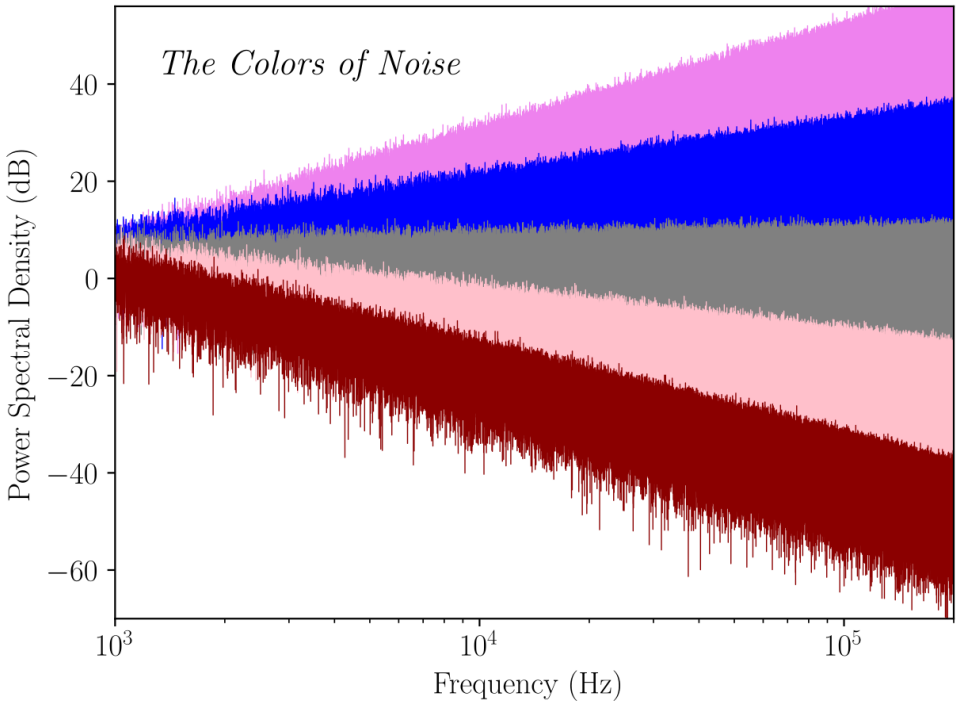
$$E = \int_{-\infty}^{\infty} |y(t)|^2 \, dt = \int_{-\infty}^{\infty} |Y(\omega)|^2 \, d\omega$$

- The *energy spectral density* of $y(t)$ is defined as

$$\bar{S}_{yy}(\omega) = |Y(\omega)|^2$$

  which makes sense since the value of $|Y(\omega)|^2 \, d\omega$ can be interpreted as a density function multiplied by an infinitesimally small frequency interval, describing the energy contained in the signal at frequency ω in the frequency interval $\omega + d\omega$.
- Note that $\bar{S}_{yy}(\omega)$ and the autocorrelation of $y(t)$ form a Fourier transform pair.

Colored noise



- White: equal power at all frequencies
- Grey: similar to white noise, but adjusted to the human ear
- Pink and red (brown): greater power at low frequencies; e.g., thunder-like sound.
- Blue and violet noise: greater power at high frequencies; e.g., the sound of a running faucet.

Low pass and band pass filters

- $F(\cdot)$: Discrete filter
- $\bar{y}_k = F(y_k)$: Filtered version of y_k
- $y_{\mathrm{d},k}$: a signal we wish to preserve (deterministic component)
- ν_k : noise component of original signal
- $\epsilon_k = F(\nu_k)$: Filtered noise component

Goal of filtering: to simultaneously achieve $\bar{y}_{\mathrm{d},k} \approx y_{\mathrm{d},k}$ **and** $|\epsilon_k| << |\nu_k|$ for all (or at least most) k.
Different implementation approaches:

- Impulse response-based - convolution
- Transfer function - difference equation
- State space - 1st order vector/matrix equation

Averaging filter

- If we only care about damping higher frequencies, an *averaging filter* may be an easy solution.
- An averaging filter is typically implemented as a discrete-time convolution between the noisy signal y_k and the filter's impulse response

$$f_{\mathrm{av},k} = \left\{ \frac{1}{m}, \frac{1}{m}, \ldots, \frac{1}{m}, 0, 0, \ldots \right\}$$

as

$$\bar{y}_k = f_{\mathrm{av},k} * y_k = \sum_{\kappa=-\infty}^{\infty} f_{\mathrm{av},\kappa} y_{k-\kappa} = \frac{1}{m} \sum_{\kappa=0}^{m-1} y_{k-m+1+\kappa}$$

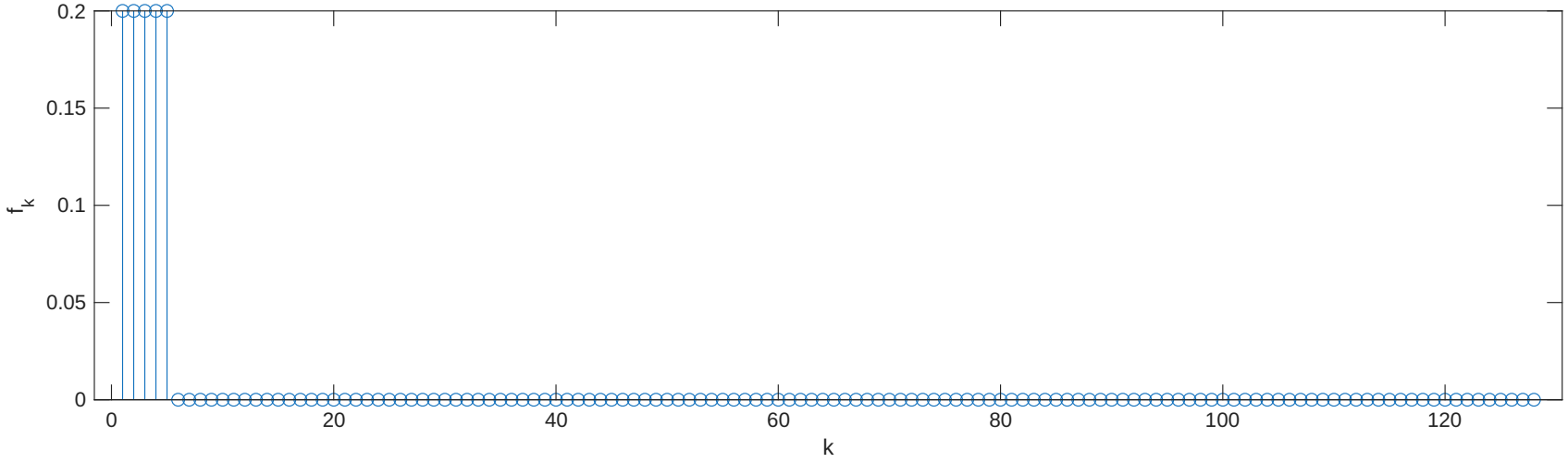
- Simple to implement—not particularly efficient.

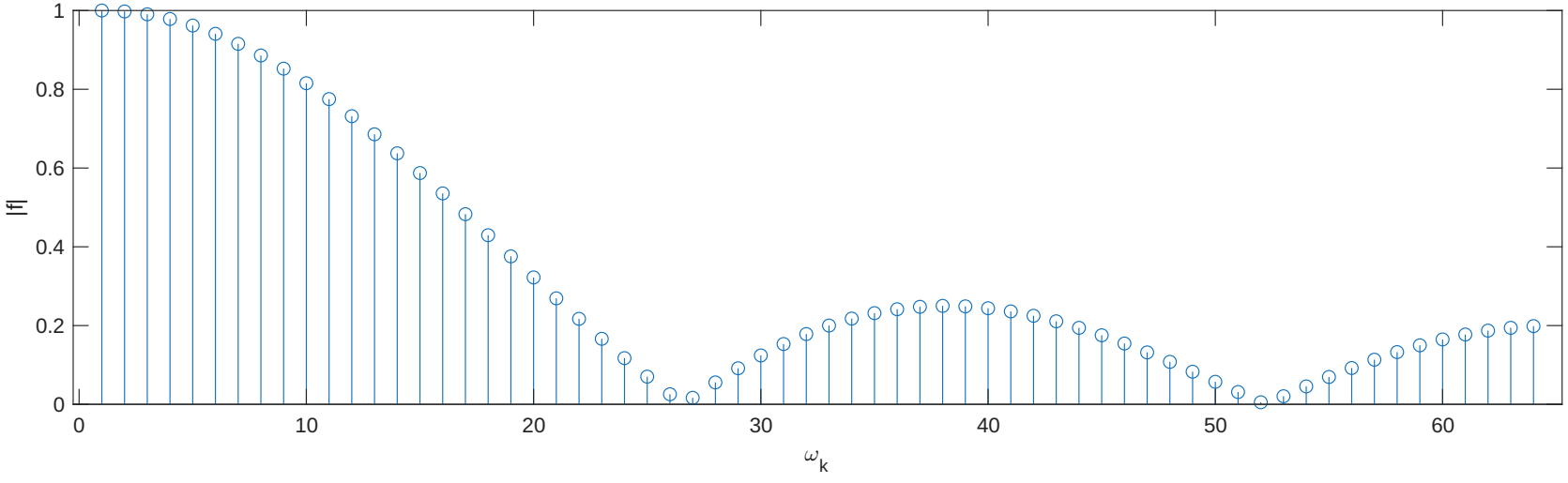
Convolution in pythonand C


```
def conv(f,y):
    N = len(y)
    n = len(f)
    y = np.zeros(N+n)
    for k in range(N+n):
        for i in range(n):
            if (k-i >= 0) and (k-i < N): # implicit zero-padding
                filtered_y[k] += f[i]*y[k-i]
    return filtered_y
```

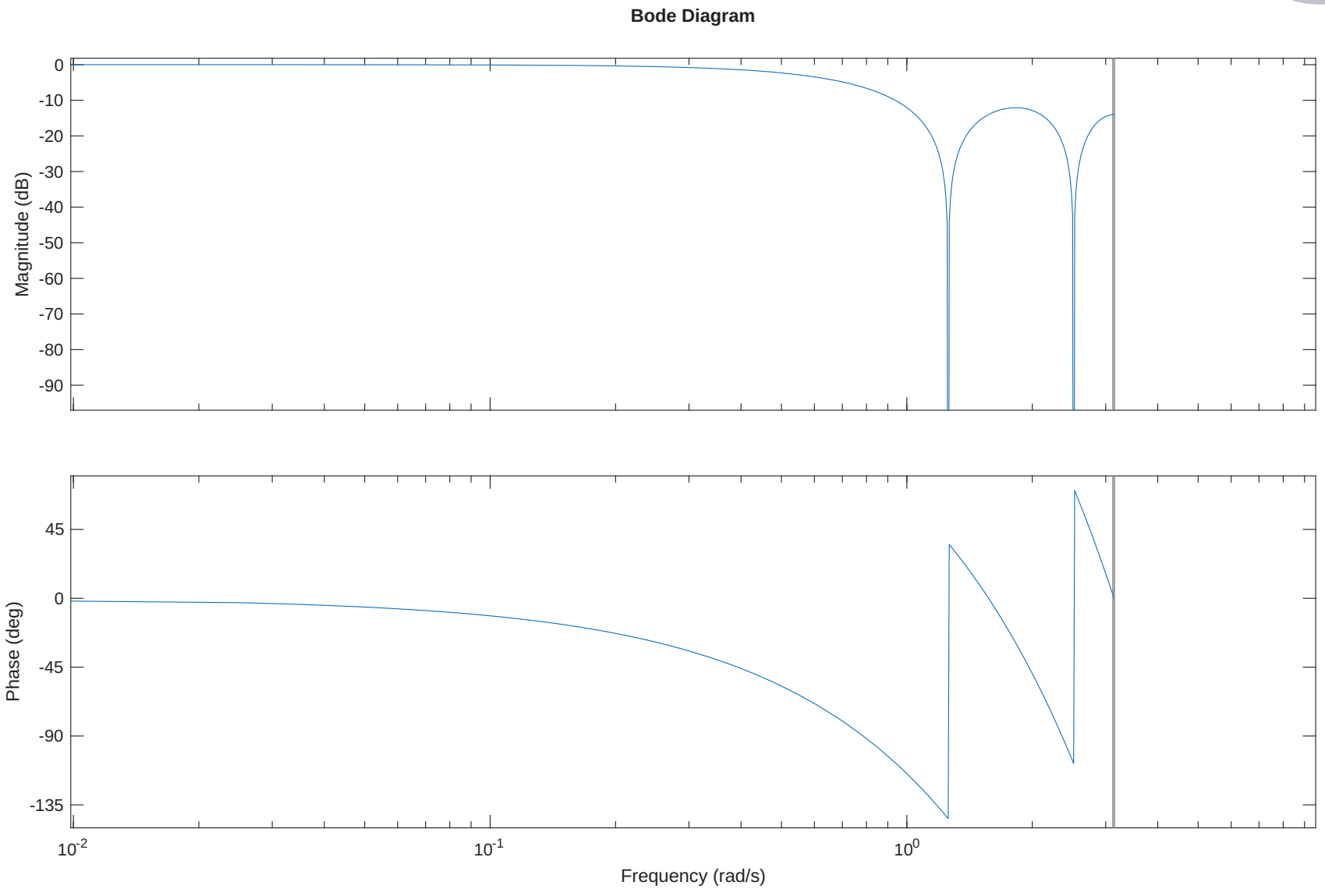
```
int conv(double *f, double *y, int N, int n, float *filtered_y) {
    // NB! the array for filtered_y should be pre-allocated, or it
    // must be allocated here using malloc!
    for (int k = 0; k < N+n; k++) {
        for (int i = 0; i < n; i++) {
            if (k-i >= 0) && (k-i < N) { // implicit zero-padding
                filtered_y[k] += f[i]*y[k-i];
            }
        }
    }
    return 0;
}
```

Averaging filter impulse and frequency response





Averaging filter bode plot



Bessel and Butterworth filters

- A Bessel filter is a type of analog linear filter with a *maximally linear phase response*,[1] which preserves the wave shape of filtered signals in the passband.
- A low-pass Bessel filter is of the general form

$$F_{\mathrm{Bes}}(s) = \frac{\psi_n(0)}{\psi_n(s/\omega_0)} \quad \text{where} \quad \psi_n(s) = \sum_{\kappa=0}^{n} \frac{(2n-\kappa)!}{2^{n-\kappa}\kappa!(n-\kappa)!} s^{\kappa}$$

- Bessel filters are difficult to design in discrete time, but can be designed in continuous time and then discretized.
- A Butterworth filter is a linear filter designed to have as flat a passband magnitude profile as possible.
- The generic form of a Butterworth filter is

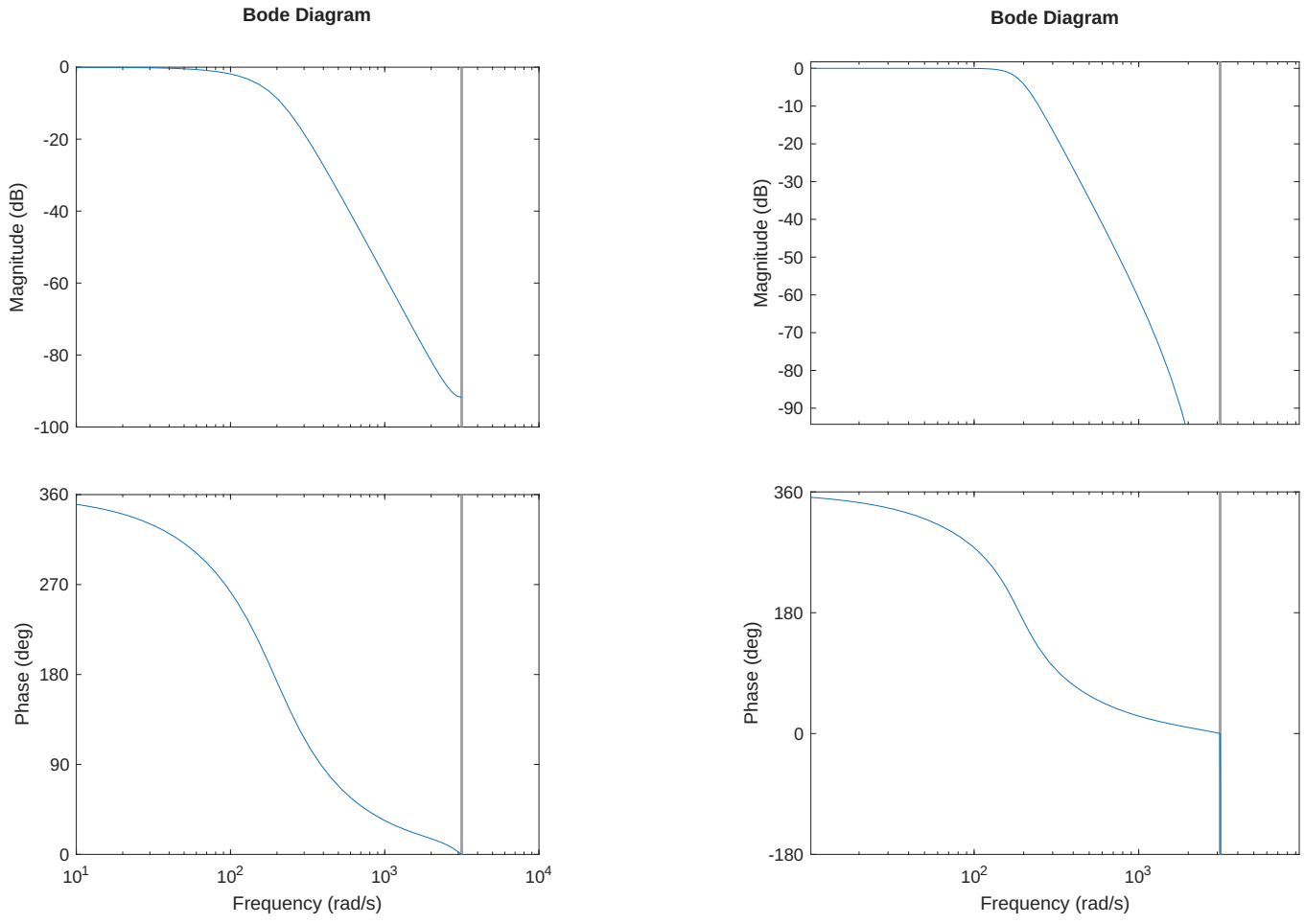
$$F_{\mathrm{But}}(s) = G_0 \prod_{\kappa=1}^{n} \frac{\omega_c}{s - s_\kappa} = G_0 \prod_{\kappa=1}^{n} \frac{\omega_c}{s - \omega_c e^{\frac{j(2\kappa+n-1)\pi}{2n}}}.$$

- Unlike the Bessel filter, here it is actually feasible to derive discretized formulae (e.g., using Tustin's method).

[1] The slope of the phase as a function of frequency, $d\phi(\omega)/d\omega$, is as close to constant as possible.

Frequency responses

Cut-off frequency 30Hz, sampling frequency 1kHz; left: Bessel, right: Butterworth



$$F_{\mathrm{Bes}}(z) = \frac{10^{-3}(0.1814z^3 + 0.6253z^2 + 0.1351z)}{z^4 - 3.43z^3 + 4.431z^2 - 2.556z + 0.555}$$

$$F_{\mathrm{But}}(z) = \frac{10^{-3}(0.06239z^4 + 0.2495z^3 + 0.3743z^2 + 0.2495z + 0.06239)}{z^4 - 3.508z^3 + 4.641z^2 - 2.743z + 0.6105}$$

Difference equation-based implementation

$$\bar{y}_k = \frac{b_m \mathrm{z}^m + b_{m-1}\mathrm{z}^{m-1} + \cdots + b_1 \mathrm{z} + b_0}{\mathrm{z}^n + a_{n-1}\mathrm{z}^{n-1} + \cdots + a_1 \mathrm{z} + a_0} y_k \quad \Rightarrow$$

$$\begin{aligned}\bar{y}_k = {} & b_m y_k + b_{m-1} y_{k-1} + \cdots + b_1 y_{k-m+1} + b_0 y_{k-m} \\ & - a_{n-1}\bar{y}_{k-1} - \cdots - a_1 \bar{y}_{k-n_1} - a_0 \bar{y}_{k-n}\end{aligned}$$

Python (but 'C-friendly') implementation:

```
def filter(b, a, y):
    yfilt = np.zeros_like(y)
    for k in range(0,len(y)-1):
        sum = 0.0
        for i in range(0,len(b)):
            if k-i > -1:
                sum = sum + b[i]*y[k-i]
        for i in range(1,len(a)):
            if k-i > -1:
                sum = sum - a[i]*yfilt[k-i]
        yfilt[k] = sum/a[0]
    return yfilt
```

$$x_{k+1} = \Phi x_k + \Gamma y_k$$
$$\bar{y}_k = H x_k + J y_k$$

where

$$\Phi = \begin{bmatrix} -a_n & -a_{n-1} & \dots & -a_1 & -a_0 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix}, \quad \Gamma = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix},$$

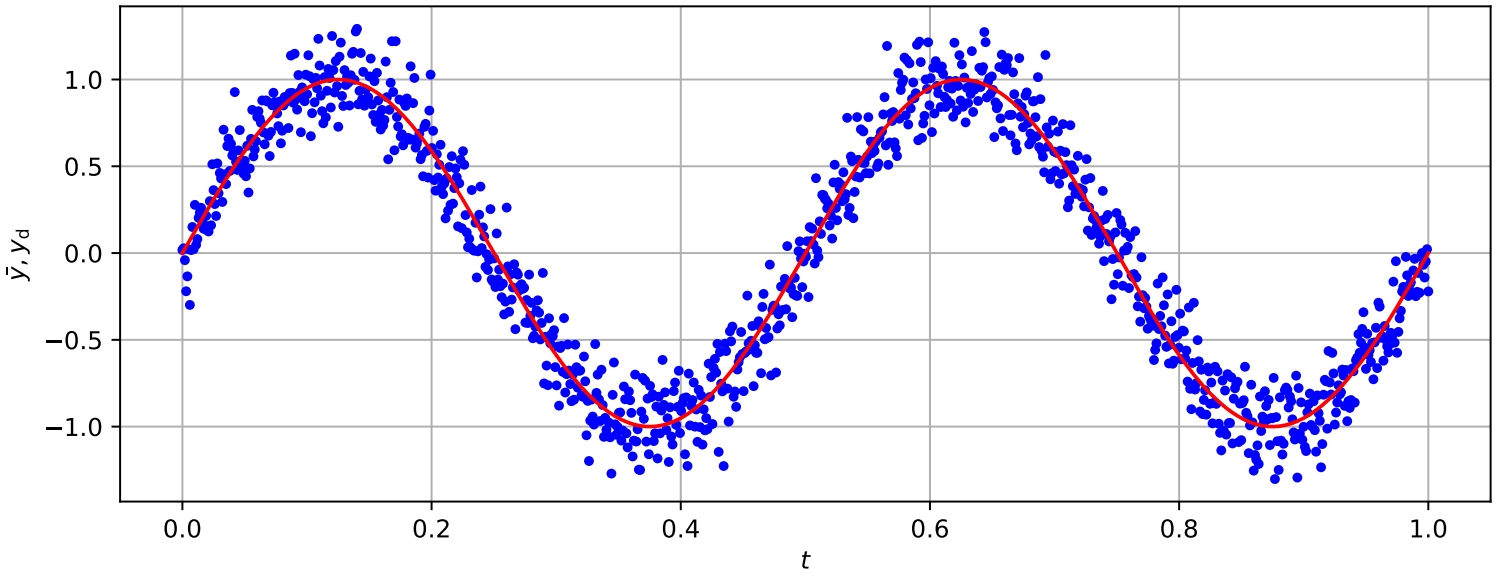
$$H = \begin{bmatrix} b_{n-1} - b_n a_{n-1} & b_{n-2} - b_n a_{n-2} & \dots & b_1 - b_n a_1 & b_0 - b_n a_0 \end{bmatrix}, \quad J = b_n$$

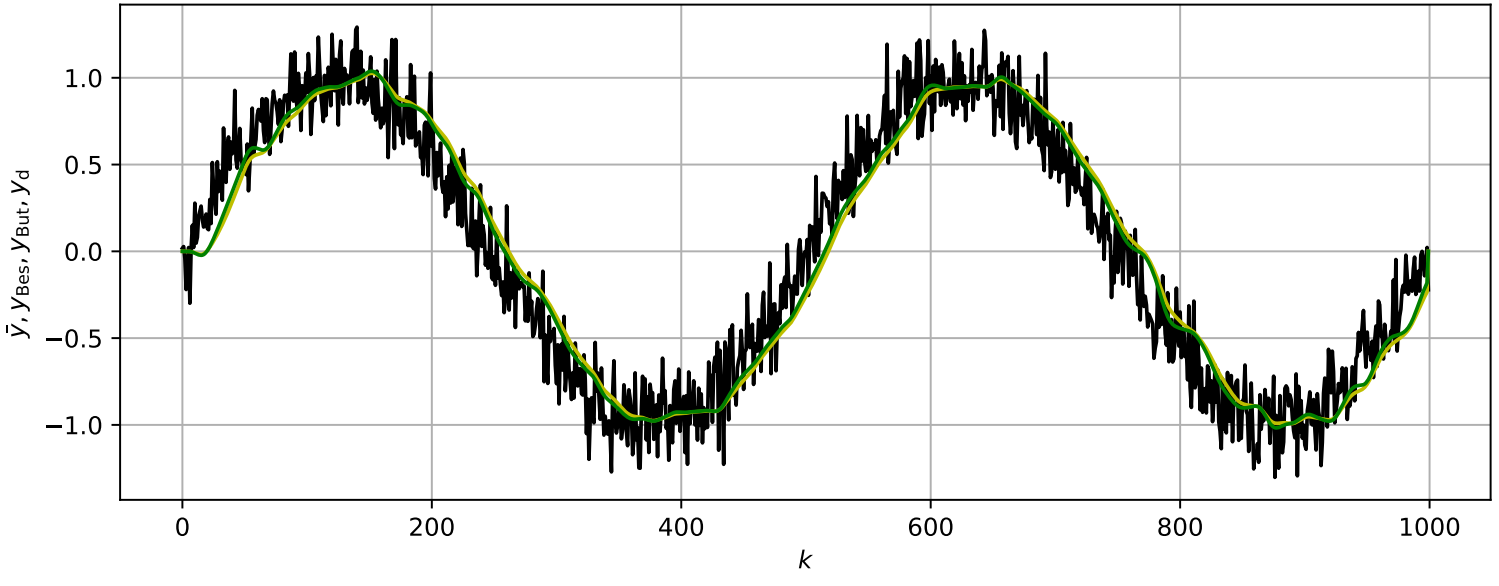
and $b_n = b_{n-1} = \cdots = b_{m+1} = 0$ whenever $m < n$.

- Obviously: when implementing filters in state space, it is advised to make use of linear algebra libraries.

Filtering performance

Red: deterministic; yellow: Bessel, green: Butterworth





Summary

- "Simple" sensors
  - Measure one continuous physical quantity;
  - Provides a simple analog or digital interface;
  - Does not require extensive processing to obtain the desired information.
- Gaussian noise: normal distributed, white noise
- Colored noise: different power at different frequency ranges, but still uncorrelated with anything—including time-shifted versions of itself
- Filtering helps reduce noise, can be done in many ways.